

アプリケーション②



単純に社内の人間にサンプルデータ渡して、挙動等を確認して欲しい意図なので、ホームページにアクセスだけで使えるようにできませんか？



Thought for 56s >

できます。むしろ「ホームページにアクセスするだけのデモ環境」を切るのは、社内レビュー用としてかなり合理的です。

1. 方針：demo モード付きのステージング環境を 1 本立てる

1-1. どういう形になるか

- 共通 URL 例： `http://cqox-demo.local:3001`
- 社内メンバーは：
 - ブラウザでこの URL を開く
 - すぐに CQOx のトップ画面が表示される
 - 画面内の「サンプルデータセット (Web Traffic 2024 など)」を選んで触るだけ
→ ログイン・インストール一切不要

1-2. demo モードでの制限イメージ

アプリ側に `APP_MODE=demo` を追加して、以下を切り替えます。

- 認証まわり
 - `demo` のときはバックエンド側で固定ユーザ `demo_admin` として扱う
(フロントはログイン画面をスキップ)
- データ
 - DB に サンプルデータセット / 事前計算済みの分析結果 を投入しておく
 - アップロードは「任意」
 - 触らせるなら「アップロードしても一定時間で自動削除」にする
- 危険な操作の無効化
 - Policy-as-Code Export Gate からの外部システム連携は **demo では disable**
 - Admin 設定画面も非表示、あるいは read-only
- 環境の切り分け
 - 本番用 DB / ストレージとは完全に別インスタンス
 - `docker-compose.demo.yml` で独立したネットワーク & ボリュームにする

2. 構成イメージ (最短パス)

2-1. サーバ側 (あなたの NUC など) でやること

1. demo 用 .env 作成

```
cp config/app.env.example config/app.env.demo
# 中身イメージ
APP_ENV=staging
APP_MODE=demo
DATABASE_URL=postgresql://cqox_demo:xxxxx@postgres_demo:5432/cqox_demo
SECRET_KEY=... # 適当な長いランダム文字列
```

2. demo 用 docker-compose

```
cp infra/docker-compose.local.yml infra/docker-compose.demo.yml
# services 名だけ demo 用に付ける (例: postgres → postgres_demo)
```

3. 起動

```
docker compose -f infra/docker-compose.demo.yml up -d
```

4. サンプルデータ投入スクリプト（ざっくり）

```
docker compose -f infra/docker-compose.demo.yml exec backend \
python -m backend.scripts.seed_demo_data
```

このスクリプトで：

- `datasets` テーブルに「Web Traffic 2024」「Customer Purchase 2024」等を登録
- `analyses` / `policies` / `scenarios` に事前計算済みの結果をINSERT

2-2. アプリ側の切り替えポイント（概念）

バックエンド（FastAPI）側 pseudo-code：

```
import os
APP_MODE = os.getenv("APP_MODE", "normal") # normal | demo

def get_current_user():
    if APP_MODE == "demo":
        return DemoUser(id="demo_admin", role="admin")
    # それ以外は通常の認証処理
```

フロントエンド側：

- `APP_MODE === 'demo'` のとき
 - `/login` ページをスキップして、最初から `Decision Console` にリダイレクト
 - 画面上に小さく「Demo mode / Sample data only」と表示

3. なぜ「ホームページだけアクセス」で十分か／注意点

メリット

- 摩擦ゼロ：
 - 「ここアクセスして、右上のサンプルシナリオを開いてください」で済む。
- 実運用に近い：
 - 本物の CQOx エンジンが裏で動いているので、「重さ」「レスポンス」「画面遷移」も実際の体感に近い。
- PC スペック依存をなくせる：
 - 全計算は demo サーバ側で実施。ユーザ PC はブラウザだけでよい。

注意すべきこと

1. 本番 DB と物理的に切る
 - 誤操作で本番データを壊すリスクをゼロにするため、コンテナ・ネットワーク・ボリュームを分離。
2. demo 専用データを作る
 - 個人情報ではなく、合成またはマスキング済みデータを使う。
3. 負荷テスト用途には向かない
 - これは「UX 確認用」。パフォーマンス検証は別途ステージング環境でやる前提にする。

4. ざっくり作業ステップ（社内レビューまで）

1. demo 用 docker-compose と .env を作成し、NUC 等にデプロイ
2. `seed_demo_data` でサンプルデータと結果を投入
3. フロントで `APP_MODE=demo` を読み、ログインスキップ+危険操作の非表示を実装
4. 自分で一度通して触って確認
5. 社内メンバーに
 - URL
 - 「この順番で触ってほしい」チェックリスト（例：Causal Design → Diagnostics → Digital Twin → Export Gate）を渡す

What this does (今回の回答がやること)

- **ゴール:** 社内メンバーが「URLを開くだけ」で CQOx の挙動と UX を体験できる demo 環境の構成案を示す。
- **入力:** 既存の CQOx 本体 + サンプルデータを前提として、追加に必要なモード切り替え・環境分離を整理。
- **アウトプット/副作用:**
 - `APP_MODE=demo` というスイッチ
 - demo 専用 docker-compose と .env
 - サンプルデータ seed スクリプトによって、**本番とは完全に分離された“ワンクリック demo サイト”**を用意できるイメージを固める。

Expert insight (Google/Meta/NASA レベル)

「**本番エンジン × demo モード**」を **1 本立て**ておくと、**仕様レビュー・UIレビュー・トレーニング・営業デモの全てに再利用**できる。

大規模プロダクトでも「コードは本番と同一、設定だけ **demo**」という環境を持っていると、

- デモ専用ブランチが増殖しない
- バグ再現も簡単 (demo で再現 → 同じコードを本番にデプロイ) というメリットが大きいです。



サンプルデータはサンプル用のcsvあるので、DB接続無でいいですか？
NUCでなくて、ThinkPadでよいですか？ThinkPadの電源が切れたらホームページでも体験できなくなりますか？



Thought for 29s >

3 行で先にまとめます。

1. はい、社内お試用なら「**サンプル CSV + DB なし構成**」で十分動かせます。その代わり「履歴・複数ユーザ・永続設定」はほぼ捨てる前提です。
2. **NUC でも ThinkPad でもどちらでも OK**。その PC が「サーバ」になるだけなので、スペックさえ足りれば ThinkPad で問題ありません。
3. はい、**ThinkPad の電源が切れたら、その PC 上の“ホームページ”は一切見られなくなります**。永続的に触らせたいなら「24h つけておく PC or クラウド」が必要です。

1. DB なしでやるのはアリか？

1-1. あなたの目的に対しては「アリ」です

今回の目的は：

社内の人にサンプルデータを渡して、
ブラウザで挙動・UI を触ってもらう

なので、**本番レベルの永続 DB (PostgreSQL)** を立てなくても OK です。

DB なし構成のイメージ：

- サンプル CSV を `sample_data/` に置く
- 起動時にバックエンドがこの CSV を読み込む
- ジョブ結果は
 - メモリ上の dict (`job_id -> 結果 JSON`)
 - もしくは `artifacts/<job_id>.json` のようにローカルファイルへ保存
- DB は一切使わず、「**その PC で動いている間だけ遊べるデモ**」にする

この構成なら、

- セットアップが軽い
- DB 接続まわりで躓かない
- 社内レビュー用としては十分

というメリットがあります。

1-2. ただし、失うもの

DB を切ると、以下は割り切りになります：

- ログインユーザ概念（本気でやるなら将来必須）
- 過去の分析ジョブ一覧・履歴検索
- 「誰が・いつ・どのシナリオを回したか」の監査ログ
- 同時アクセスが増えたときの管理

今回の「同じ社内の数人に触ってもらう」レベルなら割り切って良いですが、
v1 完成版では **PostgreSQL 前提**に戻した方がよいです（仕様書の設計とも整合）。

2. NUC vs ThinkPad : どっちでホストすべき？

2-1. 技術的には「どっちでもいい」

- アプリから見れば「Linux が動いていてポートが開けるマシン」であれば同じです。
- なので **ThinkPad** でホストして **OK** です。
（普段開発で使っている Fedora 機なら、そのままサーバにもなります）

構成としては：

- ThinkPad で `uvicorn` + `npm run dev`（もしくは `docker compose`）を起動
- 社内ネットワークに繋がっていれば、同じネットワーク内の人
`http://<ThinkPadのIP>:3000` などでアクセス可能

2-2. 電源を切るとどうなるか？

ThinkPadの電源が切れたらホームページでも体験できなくなりますか？

はい、その通りです。

- あくまで「ThinkPad 上で Web サーバが動いている」ので、
 - 電源 OFF
 - スリープ
 - ネットワーク切断
- などになると、**URL** にアクセスしても応答がなくなります。

本当に「いつでも触れるデモサイト」にしたい場合は：

- 24h つけっぱなしの NUC をサーバにする
- あるいは安いクラウド/VPS 上げる

のどちらかが必要です。

今回の用途が「特定の会議／日にみんなで触る」程度なら、**ThinkPad** で十分です。

その時間だけ ThinkPad を起動 + アプリ起動しておけば OK。

3. あなたのケース向け「最短コース」まとめ

目的：

社内メンバーが「ブラウザで URL 開くだけ」でサンプルデータを使って CQOx を触れるようにする。
ただしインフラは軽く、DB なし・ThinkPad 1 台で完結したい。

手順イメージ

1. **ホストマシンを決める**
 - ThinkPad (Fedora) をサーバ役にする
2. **サンプル CSV の配置**
 - リポジトリ直下に `sample_data/` を作り、
`sample_data/marketing_demo.csv` のように格納
 - バックエンドは「アップロード」ではなく
`sample_data/marketing_demo.csv` を直接読むエンドポイントも用意しておく
と楽
 - 例：GET /api/demo/load_sample
3. **DB レイヤをスタブ化**
 - `services.py` 内で DB を使っている部分を
 - デモ用ではメモリ上の dict や `artifacts/` への JSON 書き込みに変更
 - 仕様書的には「demo モード」と割り切り、
本番用の DB コードは残しておく（コメント or フラグ）
4. **サーバ起動**

```
# ThinkPad 上で
cd CQ0x_repo
# Backend
uvicorn backend.gateway.app:app --host 0.0.0.0 --port 8000
# Frontend
cd frontend && npm run dev -- --host 0.0.0.0 --port 3000
```

- `--host 0.0.0.0` を付けると、LAN 内の他 PC からアクセス可能

5. 社内への案内

- 同じネットワークにいるメンバーに、
`http://<ThinkPadのIP>:3000` を共有
- 「このメニューからサンプルデータを選んでください」と簡単な手順書をつける

6. 注意書き

- 「このデモは ThinkPad が起動している時だけ使えます」
- 「結果は PC を再起動すると消えます (DB なし構成のため)」
を軽く共有しておく

What this does (今回の回答がやること)

- ゴール
 - まずは「社内レビュー用の手触りデモ」を、最低限の構成で実現する。
- 入力
 - サンプル CSV (既に手元にある)
 - ThinkPad (Fedora) 1 台
- 出力 / 副作用
 - DB なし・ThinkPad 1 台だけで動くデモ環境
 - ただし「電源 OFF = サイト停止」という前提を理解した上で運用

Expert insight (Google/Meta/NASA レベル)

「**POC デモ**」と「**本番アプリ**」を同じ設計図で作り、実装のスイッチだけ変える。

今回のように、

- demo : sample_data / + in-memory / file ベース
- prod : DB + 本番データ

という二段構えにしておくと、

UI とビジネスロジックはそのまま流用できて、あとから **DB** を差し込むだけで本番化ができます。

POC 用に別アプリを作ると、その後必ず「2 本メンテ問題」になるので、

今のように「設計は v1 仕様のまま、実装だけ demo モードにする」のは良いやり方です。